Database Management Systems

Dr. Dev Narayan Yadav
Department of CSE, NIT Rourkela
Email: yadavd@nitrkl.ac.in
Mo. – 8349869748
Room – CS204

Structured Query Language

Introduction to Query Language

- ***Domain-specific language*** used for managing data held in a relational database management system.
- Declarative in nature.
- Although it is only referred as query language we can:
  - Define the structure of database.
  - Modify data
  - Specify security constraints.
  - Etc.

Structured Query Language (SQL)

- In 1970, ***Edgar F. Codd*** introduced Relational Model using ***(Relational Algebra + Tuple Relational Algebra)***
  - Theoretical model to store and retrieve data.
- Letter on IBM implemented a software for this which was named as ➔ Structured English Query Language (SEQUEL) ➔ SQL
- Oracle, Microsoft etc developed there own database systems.
- Note: Currently ➔ 90% data not structured ➔ NoSQL

Classification of Database Languages

- ***DDL:*** Set of SQL Commands which is mainly used by DBA, database engineer or application developer to create modify and delete ***database structure (but not data).***
- ***DDL*** allows the specification of information about relations including:
  - The schema for each relation.
  - The domain of values associated with each attribute.
  - Integrity constraints
  - And as we will see later, also other information such as
    - The set of indices to be maintained for each relations.
    - Security and authorization information for each relation.
    - The physical storage structure of each relation on disk.
  - ***Common Commands:*** create, alter, drop, truncate, rename, comment etc.

SDL

Classification of Database Languages

- ***DML:*** Set of SQL Commands which enables users to access or modify data.
  - ***Procedural:*** User need to specify what data are needed, and how to get those data.
  - ***Declarative:*** What data are needed.
  - ***Common Commands:*** insert, update, delete.

Classification of Database Languages

- ***DCL:*** Control access to data.
  - ***Common Commands:*** commit, grant, revoke.

Classification of Database Languages

- ***DQL:*** Allow getting data from database and imposing ordering upon it.
  - ***Common Commands:*** select, order-by etc.

Classification of Database Languages

- ***VDL:*** specify user view and their mapping to conceptual data. Size of the cell, etc.

Department of CSE, National Institute of Technology Rourkela

Domain Types in SQL

- **char(n):** Fixed length character string, with user-specified length n.
- **varchar(n):** Variable length character strings, with user-specified maximum length n.
- **Int:** Integer (a finite subset of the integers that is machine-dependent).
- **Smallint:** Small integer (a machine-dependent subset of the integer domain type).
- **numeric(p,d):** Fixed point number, with user-specified precision of p digits, with d digits to the right of decimal point. (ex., numeric(3,1), allows 44.5 to be stores exactly, but not 444.5 or 0.32)

Domain Types in SQL

- **real, double precision:** Floating point and double-precision floating point numbers, with machine-dependent precision.
- **float(n):** Floating point number, with user-specified precision of at least n digits.
- We will cover few more in the course of time.

Create Table Construct

- An SQL relation is defined using the create table command:

CREATE TABLE COMMAND	Example
create table R (A1 D1, A2 D2, …. An n (integrity-constraint1), ..., (integrity-constraintk)) Where: R ➔ name of the relation/table Ai ➔ Attribute Name. Di ➔ Domain (Datatype)	**create table** Instructor (ID char(5), Name varchar(20), Dept varchar(20), Salary(numeric(8,2))

Integrity Constraints

- Not Null
- Primary Key (Aa, A2,… An)
- Foreign Key (Am, … An) References R

Example

```
create table Instructor (
        ID char(5),
        Name varchar(20) not null,
        Dept varchar(20),
        Salary(numeric(8,2),
        primary key (ID)
        foreign key(Dept) references Department)
        )
```

- Note: primary key declaration on an attribute automatically ensures **not null**.

Updates to Relation Schema

- Adding New Column: **alter table** R **add** A D
- Dropping a Column: **alter table** R **drop** A
- Drop Table: **drop** table R
- Renaming a Column: **alter table** R **rename** A_{actual} **to** A_{new}
- Renaming a Table: **alter table** R_{actual} **rename to** R_{new}

Updates Records in Relation

- Insert Data into Table: **insert into** R (A1, A2 …) **values** (V1, V2,…);
- Update Table: **update** R **set** A 1= V1', A2 =V2' **where** Cond
- Delete from Table: **delete from** R **condition**
- Delete all records: **delete from** R.

SQL Basic

Example Relations

- Relations that we will use throughout this LAB

Example Relations

Student (Stu_ID, Stu_Name, Dept, Join_Year, Age, Email)
Instructor (Inst_ID, Inst_Name, Dept, Mont_Salary, Email)
Course (Cou_ID, Cou_Name, Inst_ID, Credit)
Enrollments (Enrol_ID, Stu_ID, Cou_ID, Grade)

Basic SQL Query

- A typical query structure.

Basic Query Structure

select A1, A2, ….
From R1, R2, …
Where C1, C2…

Result ➔ Relation

- **SELECT:** The select clause lists the attributes desired in the result of a query

Example

```
SELECT * FROM Instructor;
SELECT  Inst_Name FROM Instructor;
```


Basic SQL Query

- **FROM:** Specifies the table from which to retrieve data.

Example

```
SELECT  * FROM Instructor;
```

- **WHERE:** Filter records based on a condition

Example

```
SELECT Stu_ID, Stu_Name FROM Student WHERE  Age>20;

SELECT Stu_ID, Stu_Name FROM Student WHERE  Age>20 AND Join_Year>2020;
```

Basic SQL Query

- **DISTUNC and ALL: SQL** allows duplicates in relation as well as in query results; to forcibly eliminates the duplicates **DINSTICT** keyword can be used, to include duplicate **ALL** keyword can be used.

Example: Retrieve all unique department from Student Table

```
SELECT DISTINCT Dept FROM Student
```

Example: Retrieve all students name including duplicates

```
SELECT All Name FROM Student
```


Basic SQL Query

- **AS (Alias) :** Assigning a temporary name to a table or column

Example: Retrieve Stu_Name as Name

```
SELECT Stu_Name AS Name FROM Student
```

Example: Retrieve Student Table as R

```
SELECT R.Stu_Name, R_Stu_ID,  FROM Sudent AS R
```

Basic SQL Query

- **Arithmetic Operation:** Perform mathematical operations on numerical columns

Example: Calculating Age Next Year

```
SELECT Age,  Age + 1 AS Age_Next_Year from Student
```

Example: Instructor Salary after 10% Increment

```
SELECT Inst_ID, Month_Salary * 1.1 AS New_Salary FROM Instructor;
```

Example: A mixed example

```
SELECT Stu_Name,
        CASE
        WHEN Age<20 then Age+2
        ELSE Age +1
        END
AS Updated_Age FROM Students
```


Basic SQL Query

- **CROSS JOIN:** Combine all rows from two tables

Example: Simply perform Cartesian product between two relation

```
SELECT * FROM Student, Course
```

Or

```
SELECT * FROM Student CROSS JOIN Course
```

Example: Cross Join Between Student_Name and Course Name

```
SELECT  S.Stu_Name, C.Cou_Name FROM Student S CROSS JOIN Course C;
```

Basic SQL Query

- **STRING OPERATIONS:** Perform various string operations.

Example

```
SELECT * FROM Student WHERE Name LIKE 'A%';

SELECT * FROM Course WHERE Cou_Name LIKE '%Data%';

SELECT * FROM Student WHERE Email LIKE '%@gmail.com';

SELECT * FROM Instructor WHERE Dept LIKE 'C%e';
```


Basic SQL Query

- **SOME and LIKE:** Perform various string operations.

Example

```
SELECT * FROM Student WHERE Age > SOME (SELECT Age FROM Student
WHERE Dept = 'Computer Science');

SELECT * FROM Course WHERE Cou_Name LIKE '%AI%';

SELECT * FROM Student WHERE Email LIKE 'student%';

SELECT * FROM Instructor WHERE Dept LIKE '%Science%';
```

Basic SQL Query

- **_ (used along with like):** represents a single character.

Example: Students Whose name has A as the second latter

```
SELECT * FROM Student WHERE Name LIKE '_A%';
```

Example: Students Whose name has exactly four letters (underscore is ude here)

```
SELECT * FROM Student WHERE Name LIKE '____';
```

Example: Couse Name Starts with C ends with S and exactly of 3 characters

```
SELECT * FROM Course WHERE Cou_Name LIKE 'C_S%';
```


Basic SQL Query

- **Order By:** Sort the result set based on one or more columns

Example

```
SELECT * FROM Student ORDER BY Age ASC;
SELECT * FROM Instructor ORDER BY Month_Salary DESC;
```

Example: Course sorted first by credits (desc) and then by name (asc)

```
ELECT * FROM Course ORDER BY Credits DESC, Cou_Name ASC;

SELECT * FROM Student ORDER BY Dept ASC, Join_Year ASC;
```

Basic SQL Query

- **between:** filters records within a specific range. It is inclusive and is commonly used with numeric values, dates, text.

Example: Just Specify Range

```
SELECT * FROM Student WHERE Age BETWEEN 18 AND 25;

SELECT * FROM Course WHERE Credits BETWEEN 3 AND 5;
```

Example: Returns students whose names starts with letters A to M

```
SELECT * FROM Student WHERE Name BETWEEN 'A' AND 'M';
```


Basic SQL Query

- **Union:** combine result from two queries or relation, the column in both query must have same number, order and compatible data types.

Example: Retrieve all unique student and instructor IDs

```
SELECT Stu_ID FROM Student UNION SELECT Inst_ID FROM Instructor;
```

Example: unique course names from the Course and Enrollment tables

```
SELECT Cou_Name FROM Course UNION SELECT Cou_ID FROM Enrollment;

SELECT Cou_Name FROM Course UNION ALL SELECT Cou_ID FROM Enrollment; //
with duplicates
```

Basic SQL Query

- **Union:** combine result from two queries or relation, the column in both query must have same number, order and compatible data types.

Example: Student and Instructors Joined Before 2020

```
SELECT Stu_ID, Name, Join_Year FROM Student WHERE Join_Year < 2020
UNION
SELECT Inst_ID, NULL AS Name, NULL AS Join_Year FROM Instructor WHERE Inst_ID < 2020;
```


Basic SQL Query

- **Intersection:** combine result from two queries or relation; returns only the common records.

Example: Students Whoa re Alumni

```
SELECT Stu_ID FROM Student INTERSECT SELECT Stu_ID FROM Alumni
```

Example: Students Whoa are Instructor

```
SELECT Stu_ID FROM Student INTERSECT SELECT Inst_ID FROM Instructor;
```

Example: Students Enrolled in atleast one course and listed in student table

```
SELECT Stu_ID FROM Student INTERSECT SELECT Stu_ID FROM Enrollment;
```

Basic SQL Query

- **Intersection:** combine result from two queries or relation; returns only the common records.

Example: Couse ID exist in both Couse and Enrollment table

```
SELECT Cou_ID FROM Course INTERSECT SELECT Cou_ID FROM Enrollment;
```

Example: With duplicate (just add ALL)

```
SELECT Stu_ID FROM Student INTERSECT ALL SELECT Inst_ID FROM Instructor;
```


Basic SQL Query

- **Except:** return rows that exist in the first query but not in the second

Example: Students who are not instructor

```
SELECT Stu_ID FROM Student EXCEPT SELECT Inst_ID FROM Instructor;
```

Example: Departments that have Students but no Instructors

```
SELECT Dept FROM Student EXCEPT SELECT Dept FROM Instructor;
```

Example: Students who have not enrolled in any course

```
SELECT Stu_ID FROM Student EXCEPT SELECT Stu_ID FROM Enrollment;
```

Example: Couse not taken by any Students

```
SELECT Cou_ID FROM Course EXCEPT SELECT Cou_ID FROM Enrollment;
```

Basic SQL Query

- **Except:** return rows that exist in the first query but not in the second

Example: Students who are not instructor (Using JOIN)

```
SELECT S.Stu_ID FROM Student S LEFT JOIN Instructor I ON S.Stu_ID = I.Inst_ID WHERE
I.Inst_ID IS NULL;
```

Example: Students not enrolled in any Course (USING JOIN)

```
SELECT S.Stu_ID
FROM Student S
LEFT JOIN Enrollment E ON S.Stu_ID = E.Stu_ID
WHERE E.Stu_ID IS NULL;
```


Basic SQL Query

- **In** and **not in:** checks if a value exist/do not match in a specific list or sub-query.

Example: Find Students from specific set of Course

```
SELECT * FROM Student WHERE Stu_ID IN (SELECT Stu_ID FROM Enrollment
WHERE Cou_ID IN ('C101', 'C102', 'C103'));
```

Example: Find Students not enrolled in any course

```
SELECT * FROM Student  WHERE Stu_ID NOT IN (SELECT Stu_ID FROM
Enrollment);
```

Example: Instructor from Specific Department

```
SELECT * FROM Instructor
WHERE Dept IN ('Computer Science', 'Mathematics', 'Physics');
```

Basic SQL Query

- **In** and **not in:** checks if a value exist/do not match in a specific list or sub-query.

Example: Find course which is not assigned to an instructor

```
SELECT * FROM Course WHERE Cou_ID NOT IN (SELECT Cou_ID FROM Enrollment);
```

Example: Find students NOT enrolled in any course (left join)

```
SELECT S.* FROM Student S LEFT JOIN Enrollment E ON S.Stu_ID = E.Stu_ID WHERE
E.Stu_ID IS NULL;
```


Aggregator Functions

Example: Average age of students

```
SELECT AVG(Age) AS Average_Age FROM Student;
```

Example: Retrieve the highest and lowest instructor salaries

```
SELECT MAX(Month_Salary) AS Highest_Salary, MIN(Month_Salary) AS Lowest_Salary
FROM Instructor;
```

Example: Total Number of Students

```
SELECT COUNT(*) AS Total_Students FROM Student;
```

Example: Number of Students in each Department

```
SELECT Dept, COUNT(Stu_ID) AS Student_Count FROM Student GROUP BY Dept;
```

Aggregator Functions with Having

Example: Find departments with more than 5 students

```
SELECT Dept, COUNT(Stu_ID) AS Student_Count FROM Student GROUP BY Dept HAVING
COUNT(Stu_ID) > 5;
```

Example: Find courses with an average grade greater than 80

```
SELECT Cou_ID, AVG(Grade) AS Avg_Grade FROM Enrollment GROUP BY Cou_ID HAVING
AVG(Grade) > 80;
```


Aggregator Functions: All Combined

- (Finds total students, average age, youngest, oldest, and total age for each hostel, filtering hostels with more than 3 students, sorted by average age in descending order.

Example

```
SELECT Hostel,
        COUNT(*) AS Total_Students,
        AVG(Age) AS Average_Age,
        MIN(Age) AS Youngest,
        MAX(Age) AS Oldest,
        SUM(Age) AS Total_Age
FROM Relation
GROUP BY Hostel
HAVING COUNT(*) > 3
ORDER BY AVG(Age) DESC;
```

Nested Query

- Query inside another query:
  - Retrieve data based on the result of another query.
  - Perform calculations using aggregated results.
  - Compare values using conditions (IN, NOT IN, EXISTS, ALL, ANY).

Example: Find students older than the average student age

```
SELECT * FROM Student WHERE Age > (SELECT AVG(Age) FROM Student);
```

or

```
SELECT S.* FROM Student S JOIN (SELECT AVG(Age) AS Avg_Age FROM Student) AS Sub
ON S.Age > Sub.Avg_Age;
```


Nested Query

Example: Retrieve courses with more credits than the average credits

```
SELECT * FROM Course WHERE Credits > (SELECT AVG(Credits) FROM Course);
```

or

```
SELECT C.* FROM Course C JOIN (SELECT AVG(Credits) AS Avg_Cred FROM Course) AS
Sub ON C.Credits > Sub.Avg_Cred;
```

Example: Find students enrolled in at least one course

```
SELECT * FROM Student WHERE Stu_ID IN (SELECT DISTINCT Stu_ID FROM Enrollment);
```

Or

```
SELECT * FROM Student S WHERE EXISTS (SELECT 1 FROM Enrollment E WHERE S.Stu_ID
= E.Stu_ID);
```

Nested Query

Example: courses where all enrolled students scored above 70

```
SELECT Cou_ID FROM Enrollment GROUP BY Cou_ID HAVING MIN(Grade) > 70;
```

Or

```
SELECT Cou_ID FROM Enrollment WHERE 70 < ALL (SELECT Grade FROM Enrollment E
WHERE E.Cou_ID = Enrollment.Cou_ID);
```


Nested Query

- Finds students older than the average age, in hostels with more than 5 students, sorted by age in descending order.)

Example

```
SELECT Name, Age, Hostel
FROM Relation
WHERE Age > (SELECT AVG(Age) FROM Relation)
AND Hostel IN (SELECT Hostel FROM Relation GROUP BY Hostel HAVING COUNT(*) > 5)
ORDER BY Age DESC;
```

NULL

- **Null:** represents a missing or unknown value in SQL. (not equal to anything even another NULL)

Example: Find students whose email is missing

```
SELECT * FROM Student WHERE Email IS NULL;
```

Example: COALESCE() replaces NULL values with a default.

```
SELECT Stu_ID, COALESCE(Email, 'No Email Provided') AS Email FROM Student;
```

Example: Find students whose email is provided

```
SELECT * FROM Student WHERE Email IS NOT NULL;
```


NULL

- **Null:** represents a missing or unknown value in SQL. (not equal to anything even another NULL)

Example: Find the total number of students, considering NULL values

```
SELECT COUNT(*) AS Total_Students, COUNT(Email) AS Students_With_Email FROM
Student;
```

Department of CSE, National Institute of Technology Rourkela

Three Valued Logic

- **TRUE:** The condition is satisfied.
- **FALSE**: The condition is not satisfied.
- **UNKNOWN:** The condition involves a NULL value

Note

NULL = NULL	UNKNOWN
NULL <> 10	UNKNOWN
NULL > 5	UNKNOWN
NULL IS NULL	TRUE
NULL IS NOT NULL	FALSE

Use IS NULL instead of =
Use COALESCE() to replace NULLs
Be careful with NOT IN and NULL values
Use COUNT(Column_Name) to ignore NULLs
Use IFNULL(), NVL(), or CASE in different databases

Three Valued Logic

Example: Return Students who has not provided email address

```
SELECT * FROM Student WHERE Email = NULL; // will not return any rows.

SELECT * FROM Student WHERE Email IS NULL; // this will return
```

Example: NULL in where clause

```
SELECT * FROM Student WHERE Age > 20 OR Age = NULL; // no rows will be returned
```

Example: Similar Example

```
SELECT * FROM Student WHERE Stu_ID NOT IN (SELECT Stu_ID FROM Enrollment
WHERE Stu_ID IS NOT NULL);
```

SOME

- **SOME:** The SOME operator compares a value with a set of values returned by a subquery (works similar to any)

Example: Find students older than some students in the Computer Science department

```
SELECT * FROM Student WHERE Age > SOME (SELECT Age FROM Student WHERE Dept = 'Computer Science');
```

Example: Find courses with more credits than some other courses

```
SELECT * FROM Course WHERE Credits > SOME (SELECT Credits FROM Course WHERE Cou_ID LIKE 'C1%');
```

Example: ind students who joined before some students in the 'Math' department

```
SELECT * FROM Student WHERE Join_Year < SOME (SELECT Join_Year FROM Student WHERE Dept = 'Math');
```


SOME

- **SOME:** The SOME operator compares a value with a set of values returned by a subquery (works similar to any)

Example: SOME vs ANY (They are identical)

```
SELECT * FROM Student WHERE Age > SOME (SELECT Age FROM Student WHERE Dept = 'CS');
```

or

```
SELECT * FROM Student WHERE Age > ANY (SELECT Age FROM Student WHERE Dept = 'CS');
```

SQL Intermediate

Joins

- **Inner Join:** Returns only matching rows from both tables.

Example: Retrieve students who have enrolled in at least one course.

```
SELECT S.* FROM Student S INNER JOIN Enrollment E ON S.Stu_ID = E.Stu_ID;
```

Example: Retrieve student names and course names

```
SELECT S.Name, C.Cou_Name
FROM Student S
INNER JOIN Enrollment E ON S.Stu_ID = E.Stu_ID
INNER JOIN Course C ON E.Cou_ID = C.Cou_ID;
```

Joins

- **Inner Join:** Returns only matching rows from both tables.

Example: Find instructors who are also students (if any exist)

```
SELECT I.Inst_ID, I.Dept, S.Name
FROM Instructor I
INNER JOIN Student S
ON I.Inst_ID = S.Stu_ID;
```


Joins

- **Left Outer Join:** Returns all records from the left table and matching records from the right table. If no match, NULL appears

Example: Retrieve all students with their enrolled courses (show NULL if not enrolled)

```
SELECT S.Stu_ID, S.Name, E.Cou_ID
FROM Student S
LEFT JOIN Enrollment E
ON S.Stu_ID = E.Stu_ID;
```

Example: Retrieve all instructors along with the courses they are teaching (show NULL if no course assigned)

```
SELECT I.Inst_ID, I.Dept, C.Cou_Name
FROM Instructor I
LEFT JOIN Course C
ON I.Inst_ID = C.Inst_ID;
```

Joins

- **Right Outer Join:** Returns all records from the right table and matching records from the left table. If no match, NULL appears.

Example: Retrieve all enrollments and their corresponding student names (NULL if no student found)

```
SELECT E.Enrol_ID, S.Name, E.Cou_ID
FROM Student S
RIGHT JOIN Enrollment E
ON S.Stu_ID = E.Stu_ID;
```

Example: Retrieve all courses along with their instructors (NULL if no instructor assigned)

```
SELECT C.Cou_ID, C.Cou_Name, I.Inst_ID
FROM Instructor I
RIGHT JOIN Course C
ON I.Inst_ID = C.Inst_ID;
```


Joins

- **Full Outer Join:** Returns all records from both tables. If no match, NULL appears.

Example: Retrieve all students and all enrollments (NULL if no enrollment found)

```
SELECT S.Stu_ID, S.Name, E.Cou_ID
FROM Student S
FULL JOIN Enrollment E
ON S.Stu_ID = E.Stu_ID;
```

Example: Retrieve all instructors and their assigned courses (NULL if no course assigned)

```
SELECT I.Inst_ID, I.Dept, C.Cou_Name
FROM Instructor I
FULL JOIN Course C
ON I.Inst_ID = C.Inst_ID;
```

Joins

- **Semi Join:** Returns rows from the left table where a match exists in the right table.

Example: Retrieve students who are enrolled in at least one course

```
SELECT S.Stu_ID, S.Name
FROM Student S
WHERE EXISTS (SELECT 1 FROM Enrollment E WHERE S.Stu_ID = E.Stu_ID);
```

Example: Retrieve courses that have at least one student enrolled

```
SELECT C.Cou_ID, C.Cou_Name
FROM Course C
WHERE EXISTS (SELECT 1 FROM Enrollment E WHERE C.Cou_ID = E.Cou_ID);
```


Joins

- **Anti Join:** Returns rows from the left table where NO match exists in the right table..

Example: Retrieve students who are NOT enrolled in any course

```
SELECT S.Stu_ID, S.Name
FROM Student S
WHERE NOT EXISTS (SELECT 1 FROM Enrollment E WHERE S.Stu_ID = E.Stu_ID);
```

Example: Retrieve courses that have NO students enrolled

```
SELECT C.Cou_ID, C.Cou_Name
FROM Course C
WHERE NOT EXISTS (SELECT 1 FROM Enrollment E WHERE C.Cou_ID = E.Cou_ID);
```

Joins

Join Type	Returns
Inner Join	Matching records from both tables
Left Join	All left records + matching right records (NULL if no match)
Right Join	All right records + matching left records (NULL if no match)
Full Join	All records from both tables (NULL where no match)
Cross Join	All possible combinations of records
Semi Join	Only records from the left table that have matches in the right table
Anti Join	Only records from the left table that have NO matches in the right table

View

- A **view** is a virtual table based on a SQL query. It does not store data itself but provides a way to simplify complex queries and enhance security by restricting access to specific columns or rows.
- A view provides a mechanism to hide certain data from the view of certain users.
- Any relation that is not of the conceptual model but is made visible to a user as a "virtual relation" is called a view.
- Once a view is defined, the view name can be used to refer to the virtual relation that the view generates.

Syntax

```
CREATE VIEW view_name AS SELECT column1, column2, ... FROM table_name WHERE condition;
```

View

Example: View for Student with only Name and Department

```
CREATE VIEW Student_View AS SELECT Name, Dept FROM Student;

SELECT * FROM Student_View; // usage of the created view
```

Example: View for Instructor without salary Information

```
CREATE VIEW Faculty AS SELECT Inst_ID, Name, Dept FROM Instructor;

SELECT Name FROM Faculty WHERE Dept = 'Computer Science' // usage
SELECT * FROM Faculty WHERE Dept = 'Computer Science' // usage
```


View

Example: Department Total Salary

```
CREATE VIEW Departments_total_salary(Dept_Name, Total_salary) AS
    SELECT Dept_Name, Sum (Salary)
    FROM Instructor
    GROUP BY Dept_Name;
```

View Based on Another View

- Called nested view.

Example: View For CSE Students

```
CREATE VIEW CS_Students AS
SELECT Stu_ID, Name, Age
FROM Student
WHERE Dept = 'Computer Science';
```

Example: CSE Students Older than 20

```
CREATE VIEW CS_Students_Above_20 AS
SELECT *
FROM CS_Students
WHERE Age > 20;

SELECT * FROM CS_Students_Above_20;
```


View Based on Another View

Example: Base View for Students

```
CREATE VIEW Student_Details AS
SELECT Stu_ID, Name, Dept, Join_Year, Age FROM Student;
```

Example: Students Joined after 2020

```
CREATE VIEW Recent_Students AS SELECT * FROM Student_Details WHERE Join_Year > 2020;
```

Example: Older Students (Recently Joined)

```
CREATE VIEW Recent_Students_Above_22 AS SELECT * FROM Recent_Students WHERE Age > 22;

SELECT * FROM Recent_Students_Above_22;
```

View

Example: View for Student Enrollments with Course Names

```
// Retrieve student names along with the courses they are enrolled in.
CREATE VIEW Student_Course_View AS
SELECT S.Name, C.Cou_Name
FROM Student S
JOIN Enrollment E ON S.Stu_ID = E.Stu_ID
JOIN Course C ON E.Cou_ID = C.Cou_ID;


SELECT * FROM High_Salary_Instructors;
```


View

Example: Retrieve instructors with the courses they are Teaching.

```
CREATE VIEW Instructor_Course_View AS
SELECT I.Inst_ID, I.Dept, C.Cou_Name
FROM Instructor I
JOIN Course C ON I.Inst_ID = C.Inst_ID;


SELECT * FROM Instructor_Course_View;
```

Updating through a View

Example: Update Data through a View (the change will reflect in original table)

```
UPDATE Student_View
SET Dept = 'Mathematics'
WHERE Stu_ID = 101;

DELETE FROM Student_View
WHERE Stu_ID = 102;

INSERT INTO Student_View (Stu_ID, Name, Dept)
VALUES (105, 'Ankit', 'Physics');

UPDATE CS_Students_Above_20          // update on nested view
SET Age = 21
WHERE Stu_ID = 105;
```


Modification in View

Example: Modify a view (adding new column)

```
CREATE OR REPLACE VIEW Student_View AS
SELECT Name, Age FROM Student;

CREATE OR REPLACE VIEW Student_View AS SELECT Stu_ID, Name, Dept, Join_Year
FROM Student;

DROP VIEW Student_View;
```

View

- When Views Cannot Be Updated?
  - The view includes aggregations (SUM, AVG, COUNT, etc.).
  - The view includes DISTINCT or GROUP BY.
  - The view includes Joins on multiple tables (unless using an INSTEAD OF trigger).
  - The view includes Derived Columns (AS column_name from expressions).
  - The view is based on UNION, INTERSECT, or EXCEPT.

Materialized View

- A Materialized View (MV) is a stored result of a query that improves performance by caching data. Unlike a regular view, which is dynamically computed every time it is queried, a materialized view stores data physically in the database.
  - Faster query performance (precomputed results)
  - Reduces computation overhead for complex queries
  - Can be refreshed periodically

Syntax: Materialized View

```
CREATE MATERIALIZED VIEW view_name AS
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

Materialized View

Example: High Salaried Instructors

```
CREATE MATERIALIZED VIEW High_Salary_Instructors AS
SELECT Inst_ID, Dept, Month_Salary
FROM Instructor
WHERE Month_Salary > 250000;

REFRESH MATERIALIZED VIEW High_Salary_Instructors;


CREATE MATERIALIZED VIEW High_Salary_Instructors
REFRESH FAST ON COMMIT AS
SELECT Inst_ID, Dept, Month_Salary
FROM Instructor
WHERE Month_Salary > 5000;
```


Materialized View

Example: Using Join (Slow)

```
SELECT S.Name, C.Cou_Name
FROM Student S
JOIN Enrollment E ON S.Stu_ID = E.Stu_ID
JOIN Course C ON E.Cou_ID = C.Cou_ID;
```

Example: Using Materialized View (It will be Faster)

```
CREATE MATERIALIZED VIEW Student_Course_View AS
SELECT S.Name, C.Cou_Name
FROM Student S
JOIN Enrollment E ON S.Stu_ID = E.Stu_ID
JOIN Course C ON E.Cou_ID = C.Cou_ID;
```

Materialized View

Example: Materialized View for Recent Students

```
CREATE MATERIALIZED VIEW Fast_Recent_Students AS
SELECT * FROM Recent_Students;
```

Example: Deleting Nested View (Must be in Correct Order)

```
DROP VIEW Recent_Students_Above_22;
DROP VIEW Recent_Students;
DROP VIEW Student_Details;
```